**ПГФК ДВНЗ «УжНУ»**

# ЗВІТ

## з практичної роботи № 6

## з дисципліни «Основи програмування»

## Тема: Робота з текстовими файлами

Виконав:
студент групи КН-23
Решетарь Ерік

Перевірив:
Себастіян Володимирович

Ужгород — 2026

## 1. Мета роботи

Навчитися зберігати дані програми в текстовий файл і читати їх назад — цілим файлом через клас **File** та порядково через **StreamReader** і **StreamWriter**. Навчитися будувати шлях через **Path.Combine**, явно задавати кодування UTF-8 і коректно обробляти випадки, коли файлу немає або дані пошкоджені.

## 2. Шлях до файлу

Усі файли створюються в робочій папці програми **bin\Debug\net8.0**. Шлях формується так:

```
string file = Path.Combine(AppContext.BaseDirectory, "temperatures.txt");
```

**AppContext.BaseDirectory** повертає папку запущеної програми, а **Path.Combine** приєднує ім’я файлу й добирає правильний роздільник для Windows, Linux або macOS. Якщо вказати лише ім’я файлу, він теж потрапить у цю папку, але шлях буде неочевидним, тому в програмі він будується явно.

## 3. Лістинг програми

Код складається з окремих статичних методів, які викликаються з **Main**. Коментарів у коді немає, призначення методів зрозуміле з їхніх назв.

```
using System;
using System.Collections.Generic;
using System.Globalization;
```

```
using System.IO;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;

namespace Practice12
{
    internal class Program
    {
        const string DisplayDir = @"bin\Debug\net8.0";
        static readonly string BaseDir = AppContext.BaseDirectory;
        static string P(string name) => Path.Combine(BaseDir, name);

        static void Main()
        {
            Console.OutputEncoding = Encoding.UTF8;
            Console.WriteLine("=== Практична робота №6. Робота з текстовими файлами ===");
            Console.WriteLine($"Робоча папка: {DisplayDir}");
            Console.WriteLine();
            Task1(); Task2(); Task3(); Task4(); Task5();
            Self1(); Self2(); Self3(); Tests();
        }

        static void Task1()
        {
            Console.WriteLine("--- Завдання 1. Температури ---");
            double[] saved = { -4.5, -3.2, 1.4, 8.7, 14.6, 18.9, 20.4, 19.8, 14.2, 8.1, 2.3, -2.6
};
            string file = P("temperatures.txt");
            SaveTemperatures(file, saved);
            double[] loaded = LoadTemperatures(file);
            Console.WriteLine("Збережено: " + Format(saved));
            Console.WriteLine("Прочитано: " + Format(loaded));
            Console.WriteLine($"Кількість: {loaded.Length}, збігається:
{saved.SequenceEqual(loaded)}");
            Console.WriteLine();
        }

        static void SaveTemperatures(string path, double[] values) =>
            File.WriteAllLines(path, values.Select(v =>
v.ToString(CultureInfo.InvariantCulture)), Encoding.UTF8);

        static double[] LoadTemperatures(string path)
```

```
        {
            if (!File.Exists(path)) return Array.Empty<double>();
            var result = new List<double>();
            using (var reader = new StreamReader(path, Encoding.UTF8))
            {
                string? line;
                while ((line = reader.ReadLine()) != null)
                    if (line.Trim().Length > 0)
                        result.Add(double.Parse(line, CultureInfo.InvariantCulture));
            }
            return result.ToArray();
        }

        static string Format(double[] values) =>
            string.Join("  ", values.Select(v => v.ToString("0.0",
CultureInfo.InvariantCulture)));

        static void Task2()
        {
            Console.WriteLine("--- Завдання 2. Нотатки ---");
            string file = P("notes.txt");
            AppendNote(file, "Повторити роботу з File та StreamReader.");
            AppendNote(file, "Перевірити кодування UTF-8.");
            AppendNote(file, "Здати практичну роботу №6.");
            Console.WriteLine($"Файл: {Path.GetFileName(file)}");
            Console.Write(File.ReadAllText(file, Encoding.UTF8));
            Console.WriteLine();
        }

        static void AppendNote(string path, string text) =>
            File.AppendAllText(path, $"{DateTime.Now:dd.MM.yyyy HH:mm} -
{text}{Environment.NewLine}", Encoding.UTF8);

        static void Task3()
        {
            Console.WriteLine("--- Завдання 3. Кількість рядків ---");
            Console.WriteLine($"notes.txt: {CountLines(P("notes.txt"))}");
            string missing = P("no_such_file.txt");
            if (CountLines(missing) == -1)
                Console.WriteLine($"Файл не знайдено: {Path.GetFileName(missing)}");
            Console.WriteLine();
        }
```

```
static int CountLines(string path)
{
    if (!File.Exists(path)) return -1;
    int count = 0;
    using (var reader = new StreamReader(path, Encoding.UTF8))
    {
        string? line;
        while ((line = reader.ReadLine()) != null)
            if (line.Length > 0) count++;
    }
    return count;
}

static void Task4()
{
    Console.WriteLine("--- Завдання 4. Копія з нумерацією ---");
    string source = P("source.txt"), target = P("source_numbered.txt");
    if (!File.Exists(source))
        File.WriteAllLines(source, new[] { "Перший рядок вихідного файлу", "",
            "Третій рядок після порожнього", "Четвертий рядок" }, Encoding.UTF8);
    CopyWithNumbers(source, target);
    Console.WriteLine($"Файл: {Path.GetFileName(target)}");
    Console.Write(File.ReadAllText(target, Encoding.UTF8));
    Console.WriteLine();
}

static void CopyWithNumbers(string source, string target)
{
    if (!File.Exists(source)) return;
    using (var reader = new StreamReader(source, Encoding.UTF8))
    using (var writer = new StreamWriter(target, false, Encoding.UTF8))
    {
        string? line;
        int number = 0;
        while ((line = reader.ReadLine()) != null)
        {
            if (line.Length == 0) { writer.WriteLine(); continue; }
            writer.WriteLine($"{++number}. {line}");
        }
    }
}
```

```
        static void Task5()
        {
            Console.WriteLine("--- Завдання 5. Облік витрат ---");
            string file = P("expenses.txt");
            if (!File.Exists(file))
            {
                AddExpense(file, "03.10", "продукти", 247.80m);
                AddExpense(file, "04.10", "транспорт", 35.00m);
                AddExpense(file, "05.10", "розваги", 180.50m);
                AddExpense(file, "06.10", "продукти", 312.40m);
                AddExpense(file, "07.10", "комуналка", 560.00m);
                AddExpense(file, "08.10", "транспорт", 42.00m);
                AddExpense(file, "09.10", "продукти", 198.25m);
                AddExpense(file, "10.10", "розваги", 95.00m);
                AddExpense(file, "11.10", "комуналка", 480.00m);
                AddExpense(file, "12.10", "транспорт", 60.00m);
                File.AppendAllText(file, "13.10;продукти;багато" + Environment.NewLine,
Encoding.UTF8);
            }
            Console.WriteLine($"Файл: {Path.GetFileName(file)}");
            Console.Write(File.ReadAllText(file, Encoding.UTF8));
            Console.WriteLine($"\nСума за «продукти»: {TotalByCategory(file, "продукти"):0.00}
грн\n");
            MonthlyReport(file);
            Console.WriteLine();
        }

        static void AddExpense(string path, string date, string category, decimal amount) =>
            File.AppendAllText(path, $"{date};{category};{amount.ToString("0.00", new
CultureInfo("uk-UA"))}{Environment.NewLine}", Encoding.UTF8);

        static bool TryParseAmount(string text, out decimal value) =>
            decimal.TryParse(text.Trim().Replace(',', '.'), NumberStyles.Number,
CultureInfo.InvariantCulture, out value);

        static decimal TotalByCategory(string path, string category)
        {
            if (!File.Exists(path)) return 0m;
            decimal total = 0m;
            foreach (var (parts, ok) in ReadExpenses(path, false))
                if (ok && string.Equals(parts[1], category, StringComparison.OrdinalIgnoreCase))
                    total += decimal.Parse(parts[2].Replace(',', '.'),
CultureInfo.InvariantCulture);
            return total;
```

```
        }


        static IEnumerable<(string[] Parts, bool Ok)> ReadExpenses(string path, bool reportBroken
= true)
        {
            using var reader = new StreamReader(path, Encoding.UTF8);
            string? line;
            int number = 0;
            while ((line = reader.ReadLine()) != null)
            {
                number++;
                if (string.IsNullOrWhiteSpace(line)) continue;
                string[] parts = line.Split(';');
                bool ok = parts.Length == 3 && TryParseAmount(parts[2], out _);
                if (!ok && reportBroken) Console.WriteLine($"  Рядок {number} зіпсований:
{line}");
                yield return (parts, ok);
            }
        }


        static void MonthlyReport(string path)
        {
            if (!File.Exists(path)) { Console.WriteLine("Файл витрат не знайдено."); return; }
            var totals = new Dictionary<string, decimal>(StringComparer.OrdinalIgnoreCase);
            foreach (var (parts, ok) in ReadExpenses(path))
            {
                if (!ok) continue;
                string category = parts[1].Trim();
                decimal amount = decimal.Parse(parts[2].Replace(',', '.'),
CultureInfo.InvariantCulture);
                totals[category] = totals.TryGetValue(category, out decimal sum) ? sum + amount :
amount;
            }
            decimal total = totals.Values.Sum();
            Console.WriteLine($"{"Категорія",-14}{"Сума",10}{"Частка",9}");
            foreach (var pair in totals.OrderByDescending(p => p.Value))
                Console.WriteLine($"{pair.Key,-14}{pair.Value,10:0.00}{pair.Value * 100 /
total,8:0.0}%");
            Console.WriteLine($"{"РАЗОМ",-14}{total,10:0.00}{100.0,8:0.0}%");
        }


        static void Self1()
        {
            Console.WriteLine("--- Самостійна 1. Найдовший рядок ---");
```

```
            string file = P("text.txt");
            if (!File.Exists(file))
                File.WriteAllLines(file, new[] { "Короткий рядок",
                    "Цей рядок значно довший за попередній і має перемогти",
                    "Середній рядок тексту" }, Encoding.UTF8);
            var (number, text) = LongestLine(file);
            Console.WriteLine($"Рядок №{number}, довжина {text.Length}: {text}\n");
        }

        static (int Number, string Text) LongestLine(string path)
        {
            if (!File.Exists(path)) return (-1, string.Empty);
            int bestNumber = 0, number = 0;
            string bestText = string.Empty;
            using var reader = new StreamReader(path, Encoding.UTF8);
            string? line;
            while ((line = reader.ReadLine()) != null)
            {
                number++;
                if (line.Length > bestText.Length) { bestText = line; bestNumber = number; }
            }
            return (bestNumber, bestText);
        }

        static void Self2()
        {
            Console.WriteLine("--- Самостійна 2. Злиття файлів ---");
            string part1 = P("part1.txt"), part2 = P("part2.txt"), merged = P("merged.txt");
            if (!File.Exists(part1))
                File.WriteAllLines(part1, new[] { "Рядок з першого файлу", "Ще один рядок" },
Encoding.UTF8);
            if (!File.Exists(part2))
                File.WriteAllLines(part2, new[] { "Рядок з другого файлу" }, Encoding.UTF8);
            var missing = MergeFiles(new[] { part1, part2, P("part3_missing.txt") }, merged);
            Console.WriteLine($"Файл: {Path.GetFileName(merged)}");
            Console.Write(File.ReadAllText(merged, Encoding.UTF8));
            Console.WriteLine("Пропущені: " + (missing.Count == 0 ? "немає" : string.Join(", ",
missing)));
            Console.WriteLine();
        }

        static List<string> MergeFiles(string[] sources, string target)
        {
```

```
            var missing = new List<string>();
            using var writer = new StreamWriter(target, false, Encoding.UTF8);
            foreach (string source in sources)
            {
                if (!File.Exists(source)) { missing.Add(Path.GetFileName(source)); continue; }
                writer.WriteLine($"===== {Path.GetFileName(source)} =====");
                foreach (string line in File.ReadLines(source, Encoding.UTF8))
writer.WriteLine(line);
                writer.WriteLine();
            }
            return missing;
        }

        static void Self3()
        {
            Console.WriteLine("--- Самостійна 3. Частота слів ---");
            string file = P("words.txt");
            if (!File.Exists(file))
                File.WriteAllText(file, "Файл це набір даних. Текстовий файл зберігає текст. " +
                    "Програма читає файл і аналізує текст. Файл, файл і ще раз файл.",
Encoding.UTF8);
            WordFrequency(file);
            Console.WriteLine();
        }

        static void WordFrequency(string path)
        {
            if (!File.Exists(path)) { Console.WriteLine("Файл для аналізу не знайдено."); return;
}
            var frequency = new Dictionary<string, int>();
            foreach (string word in Regex.Split(File.ReadAllText(path,
Encoding.UTF8).ToLowerInvariant(), @"[^\p{L}\p{N}]+"))
            {
                if (word.Length == 0) continue;
                frequency[word] = frequency.TryGetValue(word, out int count) ? count + 1 : 1;
            }
            Console.WriteLine($"Різних слів: {frequency.Count}");
            foreach (var pair in frequency.OrderByDescending(p => p.Value).ThenBy(p =>
p.Key).Take(10))
                Console.WriteLine($"{pair.Key,-16}{pair.Value,5}");
        }

        static void Tests()
        {
```

```
            Console.WriteLine("--- Перевірка окремих випадків ---");
            string dir = P("tests");
            Directory.CreateDirectory(dir);
            string missing = Path.Combine(dir, "missing.txt");
            if (File.Exists(missing)) File.Delete(missing);
            Console.WriteLine($"1) Файл відсутній: CountLines = {CountLines(missing)} (очікували
-1)");
            string empty = Path.Combine(dir, "empty.txt");
            File.WriteAllText(empty, string.Empty, Encoding.UTF8);
            Console.WriteLine($"2) Порожній файл: CountLines = {CountLines(empty)} (очікували
0)");
            string broken = Path.Combine(dir, "broken.txt");
            File.WriteAllLines(broken, new[] { "01.10;продукти;100,50", "02.10;транспорт;abc",
"03.10;розваги;50" }, Encoding.UTF8);
            Console.WriteLine($"3) Зіпсований рядок: TotalByCategory = {TotalByCategory(broken,
"продукти"):0.00}");
            string trailing = Path.Combine(dir, "trailing.txt");
            File.WriteAllText(trailing, "перший" + Environment.NewLine + "другий" +
Environment.NewLine +
                "третій" + Environment.NewLine + Environment.NewLine, Encoding.UTF8);
            Console.WriteLine($"4) Порожній останній рядок: CountLines = {CountLines(trailing)}
(очікували 3)");
            try { File.ReadAllText(missing, Encoding.UTF8); }
            catch (FileNotFoundException e) { Console.WriteLine($"5) {e.GetType().Name}:
{e.Message.Replace(BaseDir, "")}"); }
            string locked = Path.Combine(dir, "locked.txt");
            try
            {
                using var stream = new FileStream(locked, FileMode.OpenOrCreate,
FileAccess.ReadWrite, FileShare.None);
                File.AppendAllText(locked, "дані", Encoding.UTF8);
            }
            catch (IOException e) { Console.WriteLine($"6) {e.GetType().Name}:
{e.Message.Replace(BaseDir, "")}"); }
        }
    }
}
```

**Пояснення вибору засобів.** Для найдовшого рядка використано **StreamReader**, бо він читає файл порядково й не завантажує його повністю в пам’ять — це важливо для великих файлів. Для підрахунку частоти слів використано **Dictionary<string, int>**, оскільки кількість різних слів заздалегідь невідома, а словник дає швидкий пошук за ключем.

## 4. Вміст створених файлів

**temperatures.txt** (перші 5 рядків):

```
-4.5
-3.2
1.4
8.7
14.6
```

**expenses.txt**:

```
03.10;продукти;247,80
04.10;транспорт;35,00
05.10;розваги;180,50
06.10;продукти;312,40
07.10;комуналка;560,00
08.10;транспорт;42,00
09.10;продукти;198,25
10.10;розваги;95,00
11.10;комуналка;480,00
12.10;транспорт;60,00
13.10;продукти;багато
```

**merged.txt**:

```
===== part1.txt =====
Рядок з першого файлу
Ще один рядок

===== part2.txt =====
Рядок з другого файлу
```

# 5. Вивід програми

## 5.1. Перший запуск (файлів ще немає)

```
=== Практична робота №6. Робота з текстовими файлами ===
Робоча папка: bin\Debug\net8.0

--- Завдання 1. Температури ---
Збережено: -4.5  -3.2  1.4  8.7  14.6  18.9  20.4  19.8  14.2  8.1  2.3  -2.6
Прочитано: -4.5  -3.2  1.4  8.7  14.6  18.9  20.4  19.8  14.2  8.1  2.3  -2.6
Кількість: 12, збігається: True

--- Завдання 2. Нотатки ---
Файл: notes.txt
13.11.2026 01:25 - Повторити роботу з File та StreamReader.
13.11.2026 01:25 - Перевірити кодування UTF-8.
13.11.2026 01:25 - Здати практичну роботу №6.
```

```
--- Завдання 3. Кількість рядків ---
notes.txt: 3
Файл не знайдено: no_such_file.txt

--- Завдання 4. Копія з нумерацією ---
Файл: source_numbered.txt
1. Перший рядок вихідного файлу

2. Третій рядок після порожнього
3. Четвертий рядок

--- Завдання 5. Облік витрат ---
Файл: expenses.txt
03.10;продукти;247,80
04.10;транспорт;35,00
05.10;розваги;180,50
06.10;продукти;312,40
07.10;комуналка;560,00
08.10;транспорт;42,00
09.10;продукти;198,25
10.10;розваги;95,00
11.10;комуналка;480,00
12.10;транспорт;60,00
13.10;продукти;багато

Сума за «продукти»: 758,45 грн

  Рядок 11 зіпсований: 13.10;продукти;багато
Категорія           Сума   Частка
комуналка        1040,00    47,0%
продукти          758,45    34,3%
розваги           275,50    12,5%
транспорт         137,00     6,2%
РАЗОМ            2210,95   100,0%

--- Самостійна 1. Найдовший рядок ---
Рядок №2, довжина 53: Цей рядок значно довший за попередній і має перемогти

--- Самостійна 2. Злиття файлів ---
Файл: merged.txt
===== part1.txt =====
Рядок з першого файлу
Ще один рядок
```

```
===== part2.txt =====
Рядок з другого файлу

Пропущені: part3_missing.txt

--- Самостійна 3. Частота слів ---
Різних слів: 13
файл                6
і                   2
текст               2
аналізує            1
даних               1
зберігає            1
набір               1
програма            1
раз                 1
текстовий           1

--- Перевірка окремих випадків ---
1) Файл відсутній: CountLines = -1 (очікували -1)
2) Порожній файл: CountLines = 0 (очікували 0)
3) Зіпсований рядок: TotalByCategory = 100,50
4) Порожній останній рядок: CountLines = 3 (очікували 3)
5) FileNotFoundException: Could not find file 'tests\missing.txt'.
6) IOException: The process cannot access the file 'tests\locked.txt' because it is being used by
another process.
```

## 5.2. Другий запуск (файли вже є)

Повторний запуск додає три нові нотатки, тому кількість рядків зростає з 3 до 6:

```
=== Практична робота №6. Робота з текстовими файлами ===
Робоча папка: bin\Debug\net8.0

--- Завдання 1. Температури ---
Збережено: -4.5  -3.2  1.4  8.7  14.6  18.9  20.4  19.8  14.2  8.1  2.3  -2.6
Прочитано: -4.5  -3.2  1.4  8.7  14.6  18.9  20.4  19.8  14.2  8.1  2.3  -2.6
Кількість: 12, збігається: True

--- Завдання 2. Нотатки ---
Файл: notes.txt
13.11.2026 01:25 - Повторити роботу з File та StreamReader.
13.11.2026 01:25 - Перевірити кодування UTF-8.
13.11.2026 01:25 - Здати практичну роботу №6.
```

```
13.11.2026 01:25 - Повторити роботу з File та StreamReader.
13.11.2026 01:25 - Перевірити кодування UTF-8.
13.11.2026 01:25 - Здати практичну роботу №6.

--- Завдання 3. Кількість рядків ---
notes.txt: 6
Файл не знайдено: no_such_file.txt

--- Завдання 4. Копія з нумерацією ---
Файл: source_numbered.txt
1. Перший рядок вихідного файлу

2. Третій рядок після порожнього
3. Четвертий рядок

--- Завдання 5. Облік витрат ---
Файл: expenses.txt
03.10;продукти;247,80
04.10;транспорт;35,00
05.10;розваги;180,50
06.10;продукти;312,40
07.10;комуналка;560,00
08.10;транспорт;42,00
09.10;продукти;198,25
10.10;розваги;95,00
11.10;комуналка;480,00
12.10;транспорт;60,00
13.10;продукти;багато

Сума за «продукти»: 758,45 грн

  Рядок 11 зіпсований: 13.10;продукти;багато
Категорія           Сума   Частка
комуналка        1040,00    47,0%
продукти          758,45    34,3%
розваги           275,50    12,5%
транспорт         137,00     6,2%
РАЗОМ            2210,95   100,0%

--- Самостійна 1. Найдовший рядок ---
Рядок №2, довжина 53: Цей рядок значно довший за попередній і має перемогти

--- Самостійна 2. Злиття файлів ---
Файл: merged.txt
```

```
===== part1.txt =====
Рядок з першого файлу
Ще один рядок

===== part2.txt =====
Рядок з другого файлу

Пропущені: part3_missing.txt

--- Самостійна 3. Частота слів ---
Різних слів: 13
файл                6
і                   2
текст               2
аналізує            1
даних               1
зберігає            1
набір               1
програма            1
раз                 1
текстовий           1

--- Перевірка окремих випадків ---
1) Файл відсутній: CountLines = -1 (очікували -1)
2) Порожній файл: CountLines = 0 (очікували 0)
3) Зіпсований рядок: TotalByCategory = 100,50
4) Порожній останній рядок: CountLines = 3 (очікували 3)
5) FileNotFoundException: Could not find file 'tests\missing.txt'.
6) IOException: The process cannot access the file 'tests\locked.txt' because it is being used by
another process.
```

# 6. Таблиця тестування

| Ситуація | Очікуваний результат | Фактичний результат |
|---|---|---|
| Файлу немає | **CountLines** повертає −1, повідомлення друкує **Main** | CountLines = −1; «Файл не знайдено: no_such_file.txt» |
| Порожній файл | 0 рядків, програма не падає | CountLines = 0 |
| Файл із зіпсованим рядком | Рядок пропущено, сума порахована з решти | «Рядок 2 зіпсований: 02.10;транспорт;abc»; сума = 100,50 |

| | | |
|---|---|---|
| Порожній останній рядок | Порожній рядок не враховується | CountLines = 3 |

## 7. Перехоплені винятки

Під час перевірки програма звертається до неіснуючого файлу та до файлу, відкритого іншим потоком. Отримано такі повідомлення:

```
FileNotFoundException: Could not find file 'tests\missing.txt'.


IOException: The process cannot access the file 'tests\locked.txt'
because it is being used by another process.
```

Перший виняток запобігає перевірка **File.Exists**, другий — блок **using**, який вчасно закриває файл. В обох випадках програма не завершується аварійно.

## 8. Контрольні питання

**1. Де створюється файл, якщо вказати лише ім’я?**

У робочій папці процесу — **bin\Debug\net8.0**. Надійніше будувати шлях через **Path.Combine(AppContext.BaseDirectory, ...)**.

**2. Чим відрізняються WriteAllText і AppendAllText?**

**WriteAllText** затирає файл і записує новий вміст, а **AppendAllText** дописує дані в кінець.

**3. Для чого потрібен блок using?**

Він закриває файл навіть при виникненні винятку, інакше наступний запис може спричинити **IOException**.

**4. Чому вказують Encoding.UTF8?**

Щоб українські літери читалися правильно на будь-якій машині, а не перетворювалися на «кракозябри».

## 9. Висновок

Я навчився записувати й читати текстові файли цілим файлом і порядково, будувати шлях через **Path.Combine** та обробляти помилки й пошкоджені рядки. Найскладнішим було коректно пропускати некоректні дані, не спотворюючи підсумки.